RV College of Engineering®

Go, change the world

NETWORK PROGRAMMING AND SECURITY (Theory & Lab)

CS362IA

UNIT 1

The Transport Layer and introduction to sockets

Port Numbers:

- All three protocols of transport layer use 16-bit integer port numbers to differentiate between UDP,TCP and SCTP processes.
- When a client wants to contact a server, the client must identify the server with which it wants to communicate. TCP, UDP, and SCTP define a group of well-known ports to identify well-known services.
  - For example, every TCP/IP implementation that supports FTP assigns the well-known port of 21 (decimal) to the FTP server. Trivial File Transfer Protocol (TFTP) servers are assigned the UDP port of 69.
- Clients, on the other hand, normally use ephemeral ports, that is, short-lived ports. These port numbers are normally assigned automatically by the transport protocol to the client.

Port Numbers:

The port numbers are divided into three ranges:

1. **The well-known ports:** 0 through 1023. These port numbers are controlled and assigned by the IANA. When possible, the same port is assigned to a given service for TCP, UDP, and SCTP. For example, port 80 is assigned for a Web server, for both TCP and UDP, even though all implementations currently use only TCP.

2. **The registered ports:** 1024 through 49151. These are not controlled by the IANA, but the IANA registers and lists the uses of these ports as a convenience to the community. When possible, the same port is assigned to a given service for both TCP and UDP. For example, ports 6000 through 6063 are assigned for an X Window server for both protocols, even though all implementations currently use only TCP.

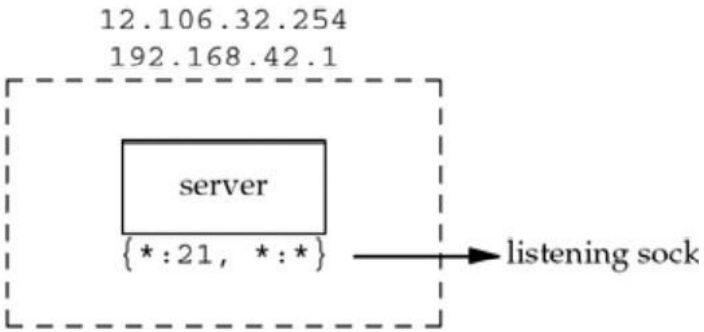
3. **The dynamic or private ports:** 49152 through 65535. The IANA says nothing about these ports. These are what we call ephemeral ports.

TCP Port Numbers and Concurrent Servers

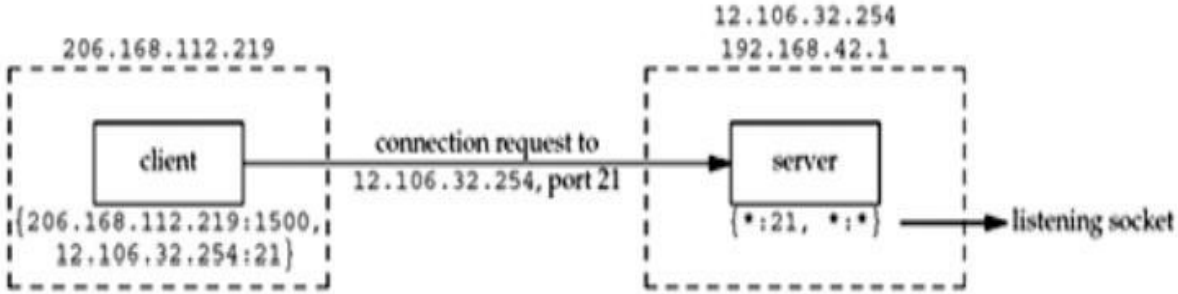
With a concurrent server, where the main server loop spawns a child to handle each new connection, what happens if the child continues to use the well-known port number while servicing a long request?

Let's examine a typical sequence. First, the server is started on the host freebsd, which is multihomed with IP addresses 12.106.32.254 and 192.168.42.1, and the server does a passive open using its well-known port number (21, for this example). It is now waiting for a client request,

1. TCP Server with a passive open on port 21

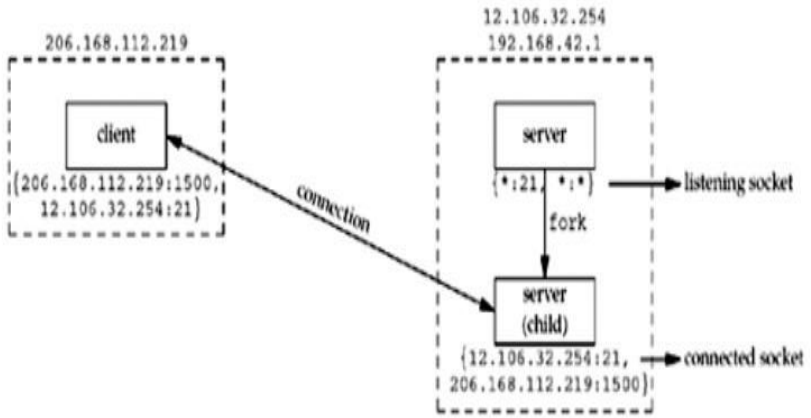


2. Connection request from client to server

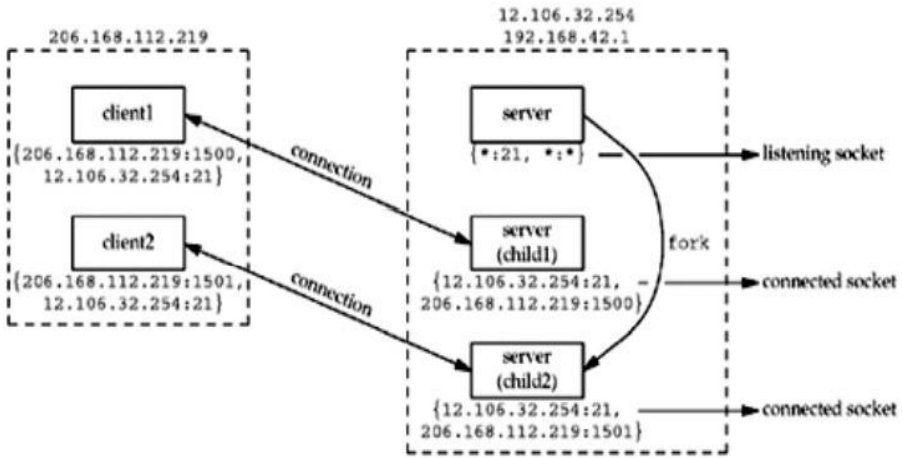


TCP Port Numbers and Concurrent Servers

3. Concurrent server has child handle client



4. Second client connection with same server



Buffer Sizes and Limitations:

Certain limits affect the size of IP datagrams:

1. The maximum size of an IPv4 datagram is 65,535 bytes, including the IPv4 header. This is because of the 16-bit total length field where for an IPv6 datagram it is 65,575 bytes, including the 40-byte IPv6 header
2. Many networks have an MTU which can be dictated by the hardware. For example, the Ethernet MTU is 1,500 bytes.
3. The minimum link MTU for IPv4 is 68 bytes.
4. The smallest MTU in the path between two hosts is called the path MTU
5. When an IP datagram is to be sent out an interface, if the size of the datagram exceeds the link MTU, fragmentation is performed by both IPv4 and IPv6. The fragments are not normally reassembled until they reach the final destination.
6. If the "don't fragment" (DF) bit is set in the IPv4 header (Figure A.1), it specifies that this datagram must not be fragmented, either by the sending host or by any router.
7. IPv4 and IPv6 define a minimum reassembly buffer size, the minimum datagram size that we are guaranteed any implementation must support.
8. TCP has a maximum segment size (MSS) that announces to the peer TCP the maximum amount of TCP data that the peer can send per segment.
9. SCTP keeps a fragmentation point based on the smallest path MTU found to all the peer's addresses.

Socket Address Structure

Definition:

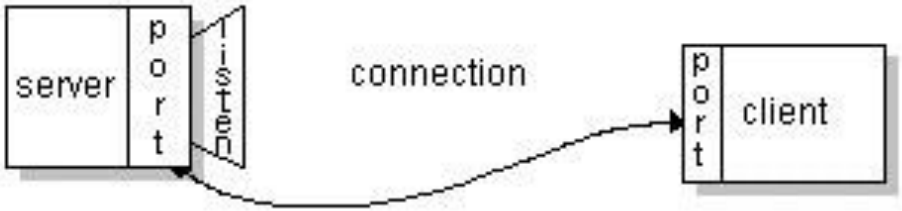
A *socket* is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to.

Client Side

1. The client knows the hostname of the machine on which the server is running and the port number on which the server is listening.
2. To make a connection request, the client tries to rendezvous with the server on the server's machine and port.
3. The client also needs to identify itself to the server so it binds to a local port number that it will use during this connection. This is usually assigned by the system.

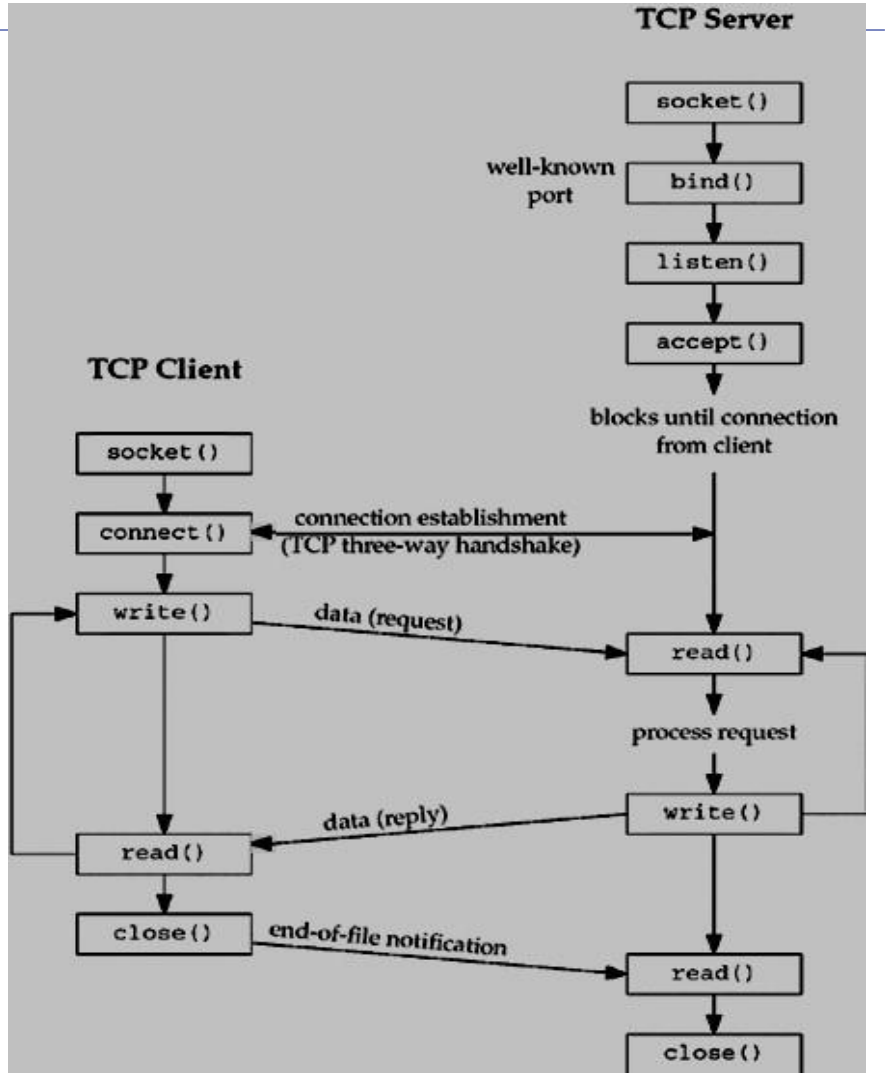
Server Side

1. The server has the name and port number defined.
2. It listens to the request and accepts connection from the client.
3. On acceptance, the server creates new socket assigns the client to it.
4. It needs a new socket so that it can continue to listen to the original socket for connection requests while attending to the needs of the connected client.



Socket Address Structure

Socket Functions- There are different set of functions at client and server side.



Socket Address Structure

1. Most socket functions require a pointer to a socket address structure as an argument.
2. Each supported protocol suite defines its own socket address structure.
3. The names of these structures begin with sockaddr_ and end with a unique suffix for each protocol suite.

IPv4 Socket Address Structure

An IPv4 socket address structure, commonly called an "**Internet socket address structure**," is named sockaddr_in and is defined by including the <netinet/in.h> header.

```
struct in_addr {
 in_addr_t s_addr;        /* 32-bit IPv4 address */
                          /* network byte ordered */
};
Struct  sockaddr_in {
uint8_t  sin_len;                /* length of structure (16) */
sa_family_t  sin_family;    /* AF_INET */
 in_port_t      sin_port;          /* 16-bit TCP or UDP port number
        network byte ordered */
struct in_addr      sin_addr;   /* 32-bit IPv4 address  network
    byte ordered */
 char sin_zero[8];                /* unused */
};
```

Socket Address Structure

- The four socket functions that pass a socket address structure from **the process to the kernel,** bind, connect, sendto, and sendmsg

all go through the sockargs function in a Berkeley-derived implementation.

- The five socket functions that pass a socket address structure **from the kernel to the process**, accept, recvfrom, recvmsg, getpeername, and getsockname,

all set the sin_len member before returning to the process.

Table: Datatypes required by the POSIX specification

Datatype	Description	Header
int8_t	Signed 8-bit integer	<sys/types.h>
uint8_t	Unsigned 8-bit integer	<sys/types.h>
int16_t	Signed 16-bit integer	<sys/types.h>
uint16_t	Unsigned 16-bit integer	<sys/types.h>
int32_t	Signed 32-bit integer	<sys/types.h>
uint32_t	Unsigned 32-bit integer	<sys/types.h>
sa_family_t	Address family of socket address structure	<sys/socket.h>
socklen_t	Length of socket address structure, normally uint32_t	<sys/socket.h>
in_addr_t	IPv4 address, normally uint32_t	<netinet/in.h>
in_port_t	TCP or UDP port, normally uint16_t	<netinet/in.h>

Generic Socket Address Structure

- A socket address structures is **always passed by reference** when passed as an argument to any socket functions.
- But any socket function that **takes one of these pointers as an argument** must deal with socket address structures from any of the supported protocol families.

```
struct sockaddr {
uint8_t  sa_len;
sa_family_t sa_family;      /* address family: AF_xxx value */
char sa_data[14];           /* protocol-specific address */

};
```

- A *generic* socket address structure is defined in the <sys/socket.h> header

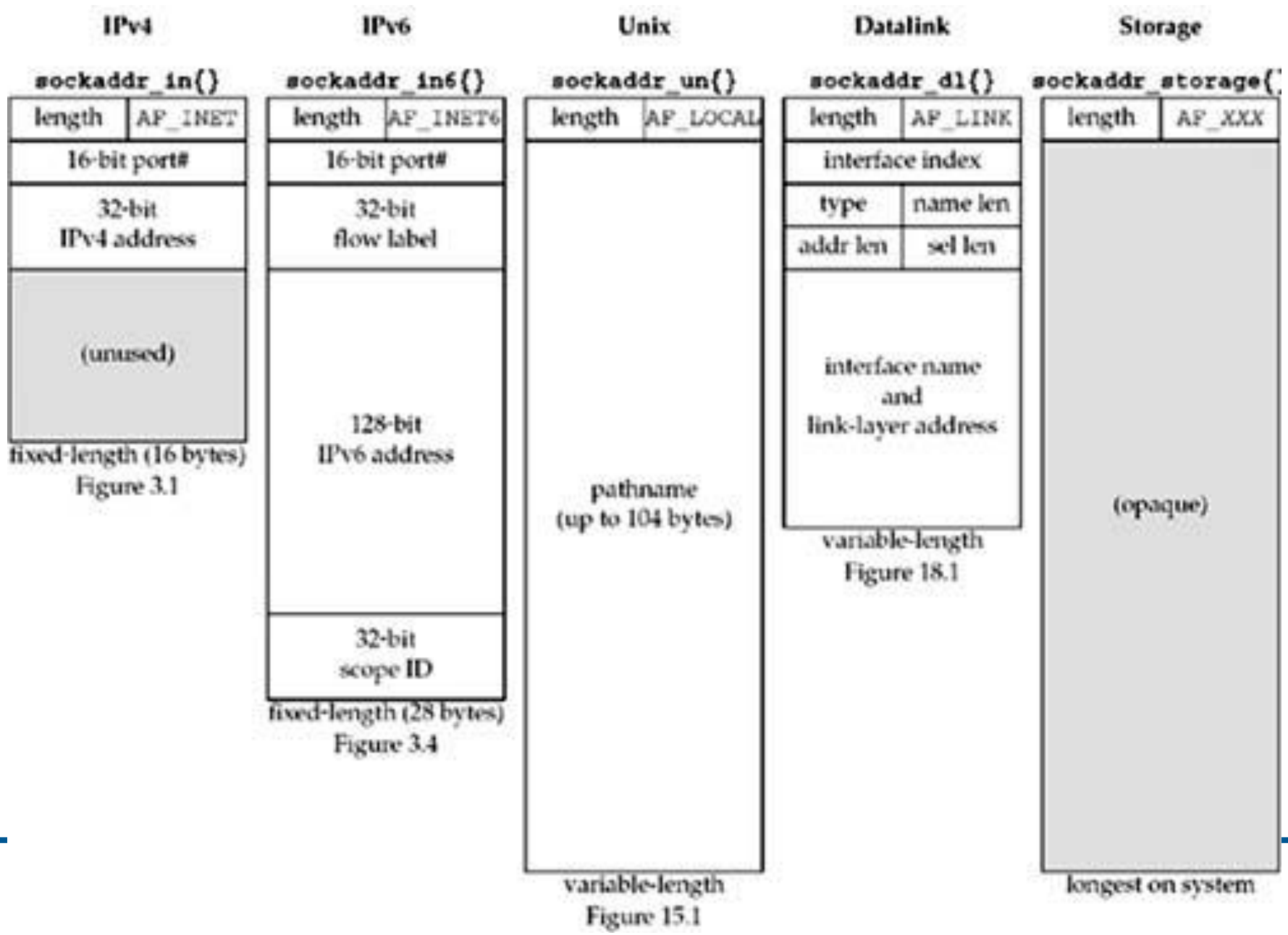
- The socket functions are then defined as taking a pointer to the generic socket address structure.
- C function prototype for the bind function:

```
int bind(int, struct sockaddr *, socklen_t);
```

- This requires that any calls to these functions must cast the pointer to the protocol-specific socket address structure to be a pointer to a generic socket address structure. For example,

```
struct sockaddr_in serv;            /* IPv4 socket address structure */
/* fill in serv{} */
bind(sockfd, (struct sockaddr *) &serv, sizeof(serv));
```

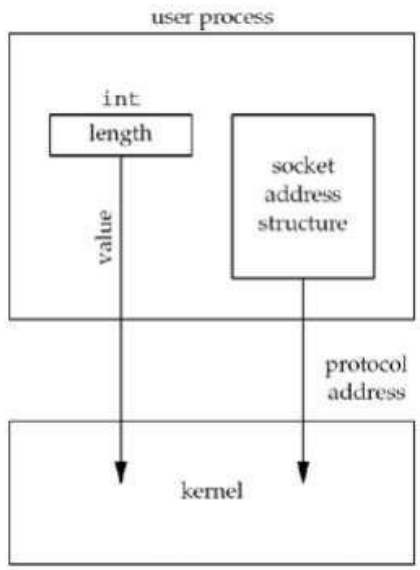
Comparison of Socket Address Structures



Value- Result Arguments

- When a socket address structure is passed to any socket function, it is always passed by reference- A pointer to the structure is passed.
- The length of the structure is also passed as an argument.
- But the way in which the length is passed depends on which direction the structure is being passed: from the process to the kernel, or vice versa.

Figure 3.7. Socket address structure passed from process to kernel.

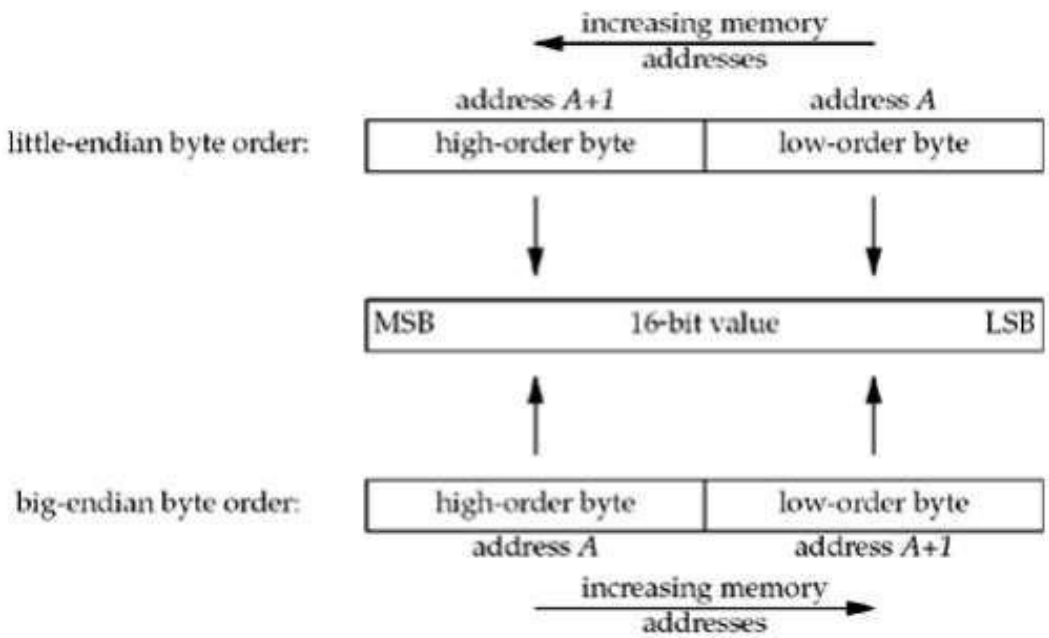


Since the kernel is passed both the pointer and the size of what the pointer points to, it knows exactly how much data to copy from the process into the kernel.

Byte Ordering Functions

- Consider a 16-bit integer that is made up of 2 bytes.
- There are two ways to store the two bytes in memory: with the low-order byte at the starting address, known as **little-endian byteorder**, or with the **high-order byte** at the starting address, known as big-endian byte order.

Figure 3.9. Little-endian byte order and big-endian byte order for a 16-bit integer.



Byte Ordering Functions

- The terms "little-endian" and "big-endian" indicate **which end** of the multibyte value, the little end or the big end, **is stored at the starting address** of the value.
- **Host byte order** refer to the byte ordering used by a given system.
- Networking protocols must specify a **network byte order**.
- The sending protocol stack and the receiving protocol stack must agree on the order in which the bytes of these multibyte fields will be transmitted.
- The **Internet protocols use big-endian byte ordering** for these multibyte integers.

Conversion Functions - 1

We use the following four functions to convert between these two byte orders:

unp_htons.h

```
#include <netinet/in.h>

uint16_t htons(uint16_t host16bitvalue);
uint32_t htonl(uint32_t host32bitvalue);

/* Both return: value in network byte order */

uint16_t ntohs(uint16_t net16bitvalue);
uint32_t ntohl(uint32_t net32bitvalue);

/* Both return: value in host byte order */
```

- `h` stands for *host*
- `n` stands for *network*
- `s` stands for *short* (16-bit value, e.g. TCP or UDP port number)
- `l` stands for *long* (32-bit value, e.g. IPv4 address)

Conversion Functions - 2

inet_aton, inet_addr, and inet_ntoa Functions

- These functions convert Internet addresses between ASCII strings (what humans prefer to use) and network byte ordered binary values (values that are stored in socket address structures).
- inet_addr does the same conversion, returning the 32-bit binary network byte ordered value as the return value. The problem with this function is that all 232 possible binary values are valid IP addresses (0.0.0.0 through 255.255.255.255), but the function returns the constant INADDR_NONE (typically 32 one-bits) on an error.
- Today, inet_addr is deprecated and any new code should use inet_aton instead.

unp_inet_aton.h

```
#include <arpa/inet.h>

int inet_aton(const char *strptr, struct in_addr *addrptr);
/* Returns: 1 if string was valid, 0 on error */

in_addr_t inet_addr(const char *strptr);
/* Returns: 32-bit binary network byte ordered IPv4 address; INADDR_NONE if error */

char *inet_ntoa(struct in_addr inaddr);
/* Returns: pointer to dotted-decimal string */
```

Conversion Functions -3

inet_pton and inet_ntop Functions

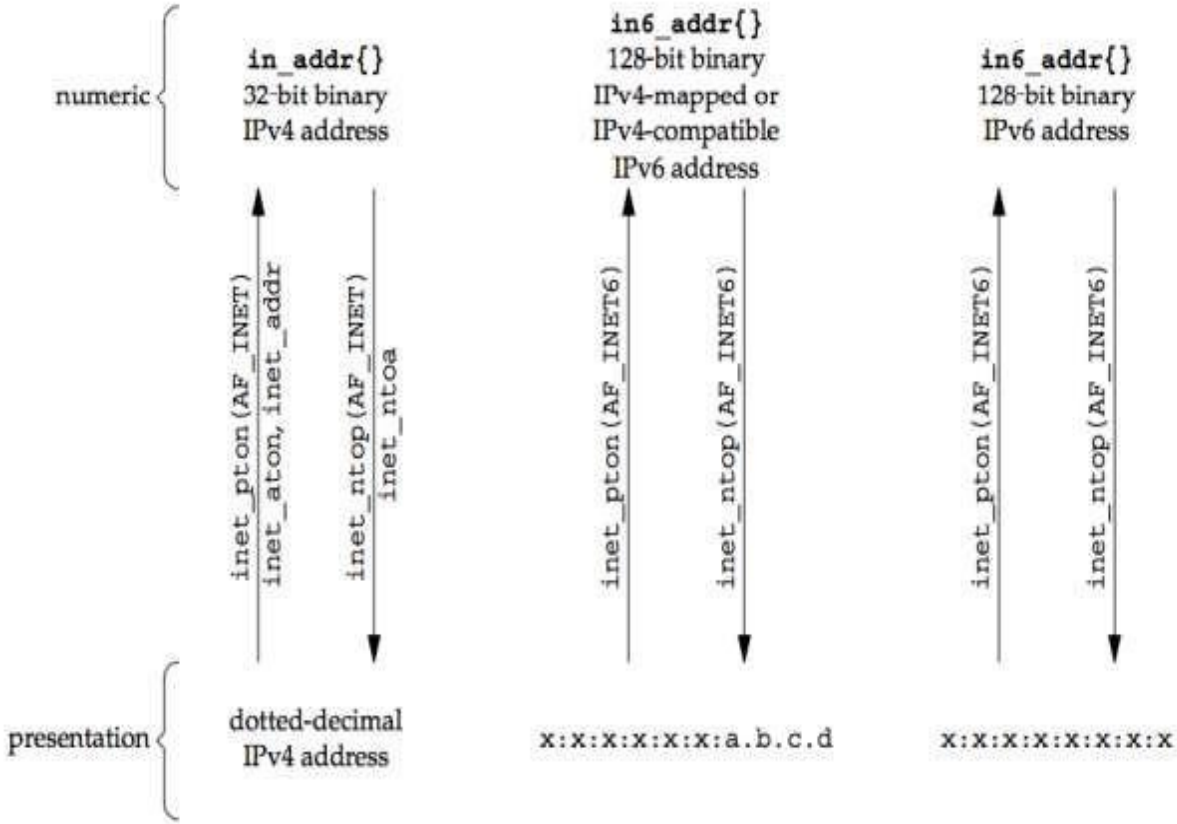
- These two functions are new with IPv6 and work with both IPv4 and IPv6 addresses. The letters "p" and "n" stand for presentation and numeric.

```
#include <arpa/inet.h>

int inet_pton(int family, const char *strptr, void *addrptr);
/* Returns: 1 if OK, 0 if input not a valid presentation format, -1 on error */

const char *inet_ntop(int family, const void *addrptr, char *strptr, size_t len);
/* Returns: pointer to result if OK, NULL on error */
```

Conversion Functions -3

- inet_pton: converts the string pointed to by strptr, storing the binary result through the pointer addrptr. If successful, the return value is 1. If the input string is not a valid presentation format for the specified family, 0 is returned.
- inet_ntop does the reverse conversion, from numeric (addrptr) to presentation (strptr).
- len argument is the size of the destination.

Conversion Functions



Byte Manipulation Functions

- There are <u>two groups of functions that operate on multibyte fields</u>, without interpreting the data, and without assuming that the data is a null-terminated C string.
- We need these types of functions when dealing with socket address structures because we need to manipulate fields such as IP addresses, which can contain bytes of 0, but are not C character strings.
- The **<u>first group of functions</u>**, whose names begin with **b (for byte)**, are from 4.2BSD and are still provided by almost any system that supports the socket functions.
- The **<u>second group of functions</u>**, whose names begin with **mem (for memory)**, are from the ANSI C standard and are provided with any system that supports an ANSI C library.

Berkeley-derived functions

- *bzero* sets the specified number of bytes to 0 in the destination. We often use this function to initialize a socket address structure to 0.
- *bcopy* moves the specified number of bytes from the source to the destination.
- *bcmp* compares two arbitrary byte strings. The return value is zero if the two byte strings are identical; otherwise, it is nonzero.

`#include <strings.h>`
`void bzero(void *dest, size_t nbytes);`
`void bcopy(const void *src, void *dest, size_t nbytes);`
`int bcmp(const void *ptr1, const void *ptr2, size_t nbytes);`
Returns: 0 if equal, nonzero if unequal

The ANSI C functions:

- *memset* sets the specified number of bytes to the value c in the destination.
- *memcpy* is similar to *bcopy*, but the order of the two pointer arguments is swapped.

`#include <string.h>`
`void *memset(void *dest, int c, size_t len);`
`void *memcpy(void *dest, const void *src, size_t nbytes);`
`int memcmp(const void *ptr1, const void *ptr2, size_t nbytes);`
Returns: 0 if equal, <0 or >0 if unequal (see text)